

Electron frontend: RAM consumption

Track N Race · Electron 43.5 · measured 7–9 October 2026

Summary

- **Electron runs as four processes, and all four count.** These are the renderer (the page), the GPU process, the main process (Node and the native engine) and a network service. Every total here is their sum.
- **Its fixed cost is about 280 MB with no session and about 505 MB live with charts on screen.** Electron itself (a blank window) is 150 MB of that.
- **About 45–63 MB more can go with three small changes** (section 7, measured twice): no depth buffer on chart canvases, a smaller V8 young generation and the network service in the main process. The engine steps in section 8 would take another 50–100 MB off a long session.
- **End of the race, All Laps on every page: 640–810 MB through the tour, 752 MB at the end, peak 1,115 MB** while the seek's history is decoded. Qt on the same run: 570 MB, peak 712 MB.
- **A whole race live on today's build: 714 MB at the end, peak 748 MB** (section 5).
- **The 103-minute live session from 7 October reached 1.2 GB**, with the renderer stuck near 575 MB from minute 40. That was before the renderer work, with React Scan on.
- **The renderer work cut the renderer by about 40 %.** At lap 11 it went from 597 to 369 MB, with React Scan on. Column tables went from 52.9 to 19.1 MB and a 55-minute race history from 79.8 to 37.5 MB.

Windows 11, RTX 4050 laptop GPU, 1920 × 1080 at 125 %. Data: the 92-minute São Paulo race, as the UDP capture (`udp_capture.bin`, replayed with `play_udp.py`) or as its recording (`udp_capture_sao_paulo_v6_2026-10-07.tnrd`). **Private memory** is memory only this program uses; it is the number to compare. The **working set** is what currently sits in physical RAM, and Windows trims it freely.

1. The 103-minute live session (before the renderer work)

A 103-minute live session, logged by the app's RAM log every 10 s (`ram_usage (2).log`). It predates the renderer work, and React Scan, a debug tool that holds old data, was on. It shows where the memory was and what started the work, not today's live figures.

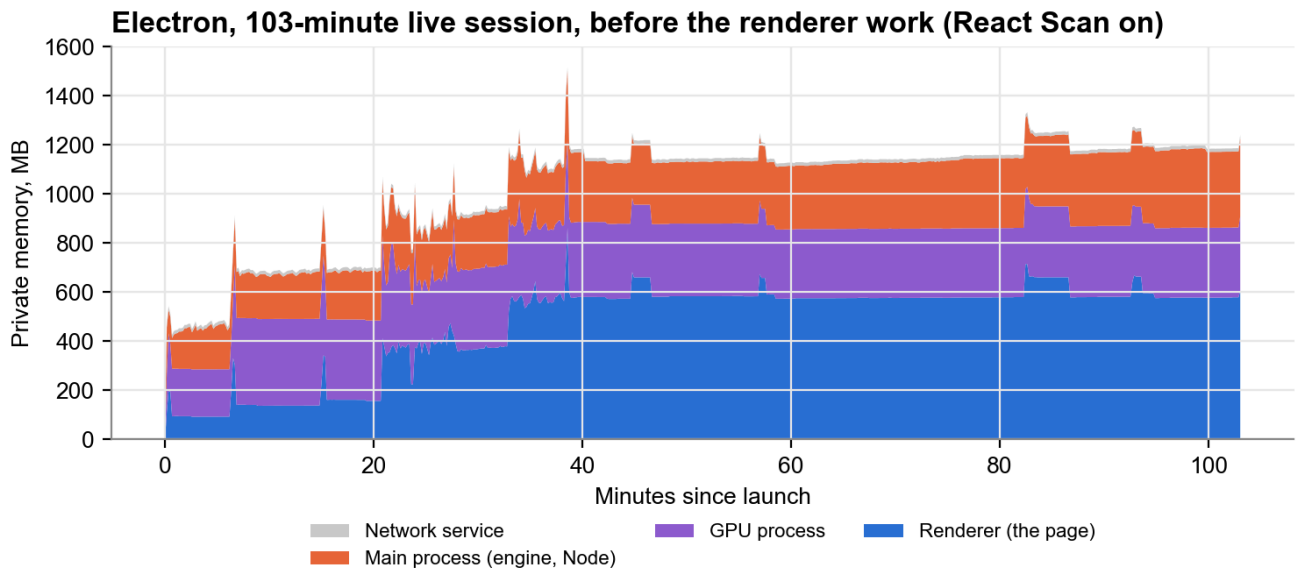


Figure 1. Private memory by process through the session.

At	Total	Renderer	GPU	Main	Network
Minute 5	476	92	193	178	13
Minute 10	678	137	354	174	13
Minute 20	700	157	328	203	13
Minute 30	926	368	327	218	13
Minute 40	1,183	580	306	284	13
Minute 60	1,128	573	282	260	13
Minute 100	1,184	577	285	310	13

The renderer jumped in steps (minutes 21, 33, 38) and then stayed near 575 MB. The investigation that followed (on a 1x replay) found that most of it was garbage V8 kept committed to keep up with 270 MB/s of allocation, plus old data React Scan held on to. The app's own telemetry was about 25 MB.

2. The renderer work

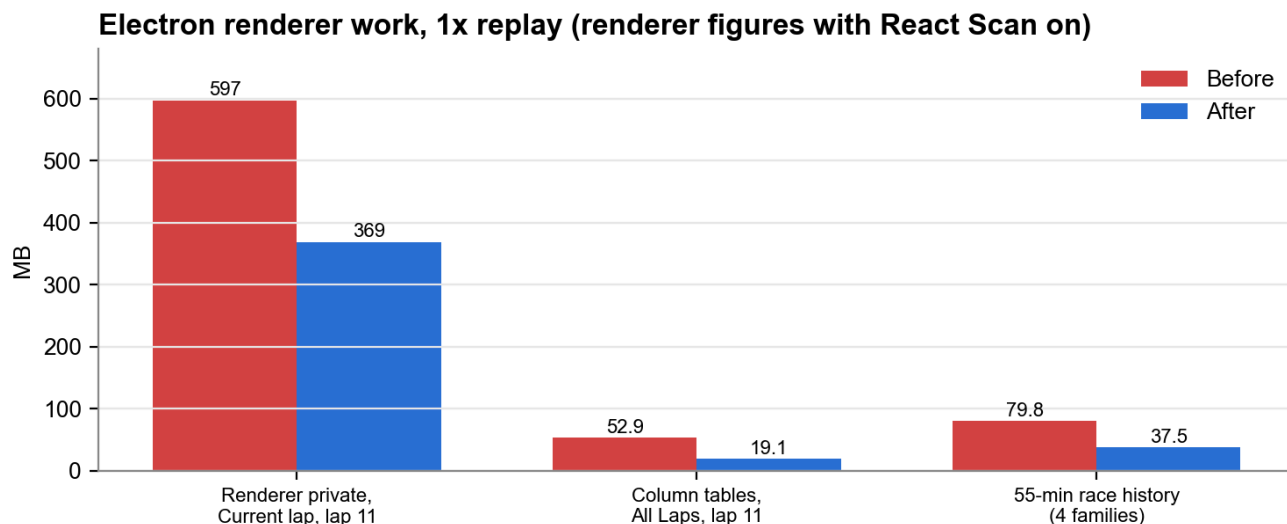


Figure 2. Measured at lap 11 of a 1x replay. The renderer-private pair was measured with React Scan on in both runs. With it off, the renderer after the work was 239 MB.

Stage	What	Effect
A	Stop rebuilding the lap-progress map on every render	Allocation 270 → 56 MB/s (React Scan on), 18–25 MB/s off
B	Honest diagnostics: renderer heap in the RAM log, real storage bytes	The figures in this report
C	Chunked Float32 column storage, history decode in a worker, binary live laps	Tables about 2.8× smaller

3. End of the race, All Laps

The recording opened, seeked to minute 88 and played with All Laps on every page, visiting Overview, Input, Power, Misc, Tyres, Trends, Overview and Analysis for about 41 s each. These are the same recording, seek point and pages as the Qt run.

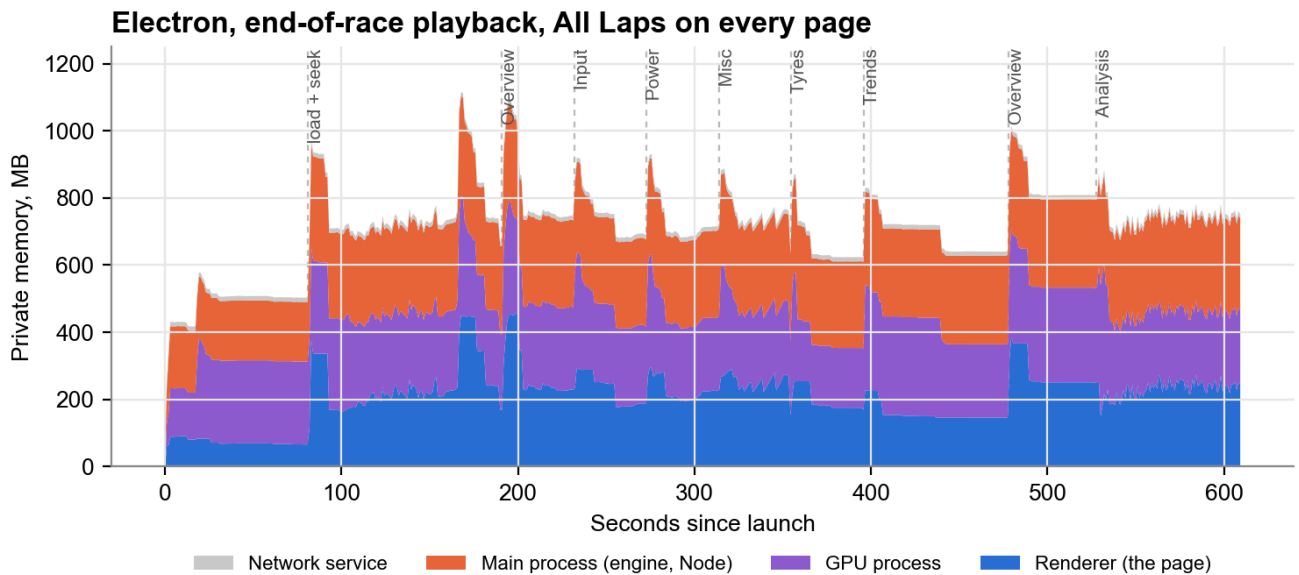


Figure 3. Private memory by process. The first tall peak (1,115 MB, at 168 s) is the seek, while the renderer decodes the 35 MB history payload.

At	Total	Renderer	GPU	Main	Network
Before the recording loaded (30–80 s)	506	68	247	178	13
Input page (250 s)	757	248	238	258	13
Misc page (350 s)	763	269	223	259	13
Trends page (420 s)	720	151	293	263	13
End of the tour (600 s)	752	244	221	274	13

4. Against Qt

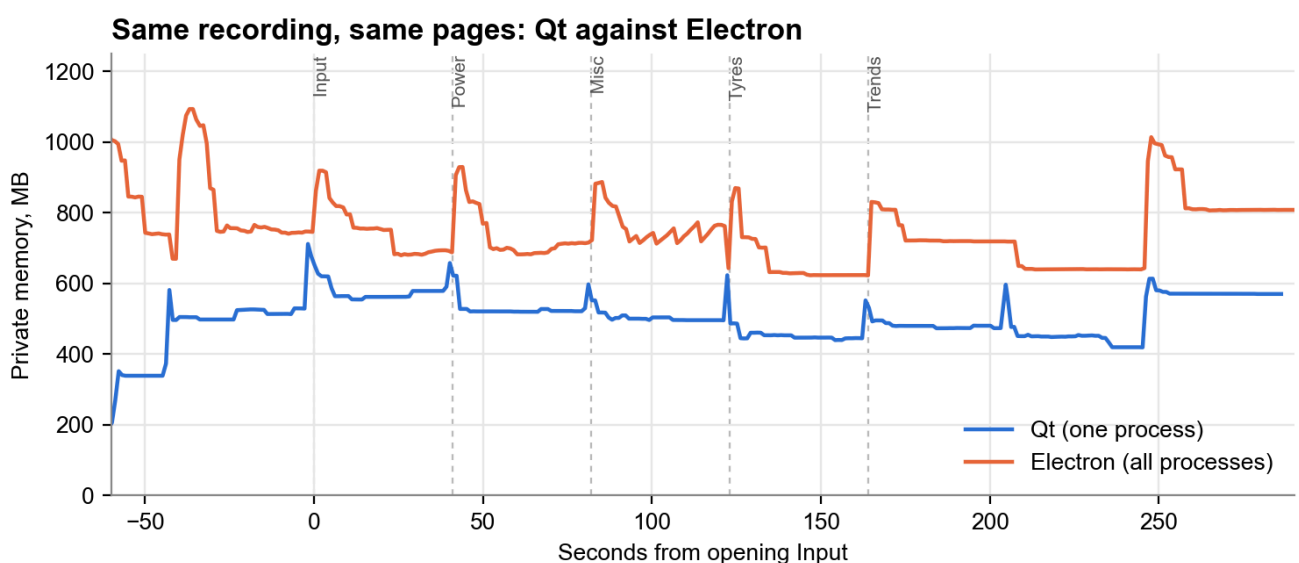


Figure 4. The two apps on the same tour, aligned at the moment each opens Input. Qt is one process; Electron's line is all four.

Electron sits 150–250 MB above Qt on every page, and its page-switch peaks are 200–300 MB taller. With no session the gap is about 95 MB (Electron 282 MB, Qt 187 MB); once charts are live it is about 250 MB.

5. A whole race on today's build

On 9 October today's build ran the whole capture live at 1x, fresh from launch: 99 minutes, the formation lap and 71 laps, on Overview with Current Lap, React Scan off, recording the race to a file as it does by default.

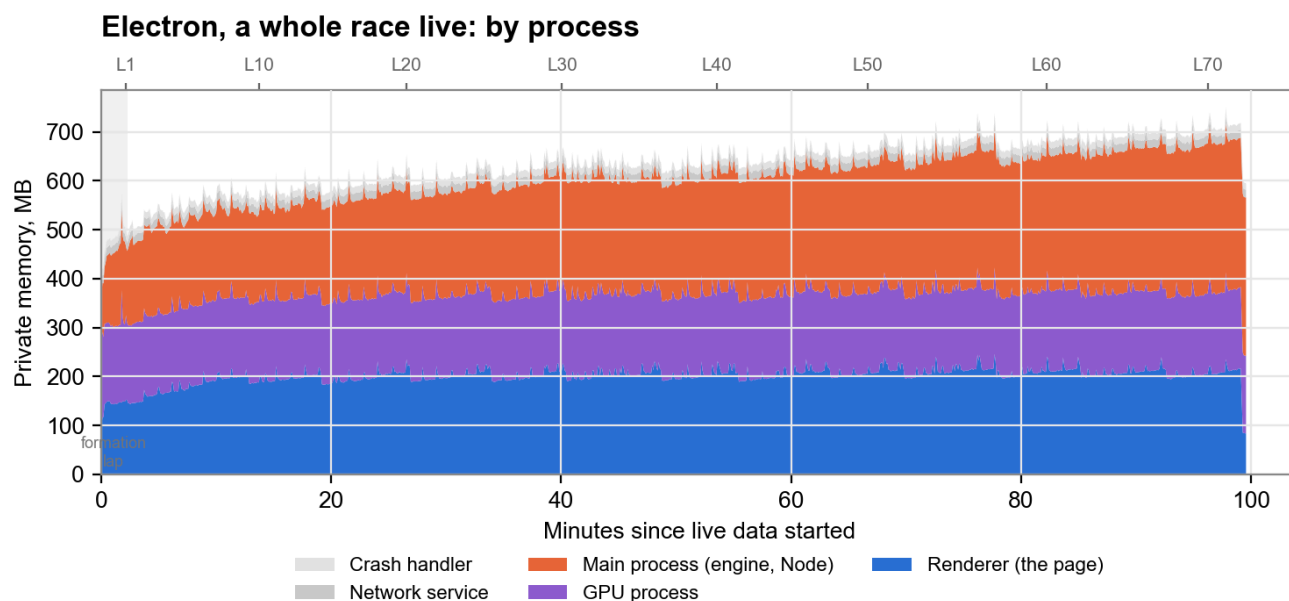


Figure 5. Private memory by process over the whole race. Lap numbers run along the top. The main process, which runs the engine, carries most of the growth (+140 of +214 MB).

	Total	Main	GPU	Renderer	Network	Crash handler
Minute 2	502	166	159	148	16	13
Minute 15	573	188	165	191	16	13
Minute 30	603	213	164	197	16	13
Minute 60	644	250	163	202	16	13
Minute 90	697	291	167	209	16	13
End of the race	714	306	164	214	16	13

Electron ended the race at 714 MB, with a peak of 748 MB, and grew about 80 MB an hour over the last hour. The main process went from 166 to 306 MB: the engine, including its in-memory copy of the recording (section 8, step 1). The renderer settled at 190–215 MB after about 15 minutes, and the GPU process stayed flat near 165 MB. The 7 October session in section 1 reached 1.2 GB on the old build with React Scan on; today's build ends a whole race 40 % lower.

6. What's proven and what isn't

Claim	Status
End of the race, All Laps: 640–810 MB through the tour, peak 1,115 MB	Measured (8 October)
No session: 282 MB; live with charts after 100 s: about 505 MB	Measured (9 October; 4 live baseline runs)
Each change in section 7	Measured in two sets of runs; totals vary ± 10 MB, per-process figures less
The 103-minute session reached 1.2 GB with the renderer near 575 MB	Measured, but on the old build with React Scan on
Renderer 597 → 369 MB at lap 11 after the renderer work	Measured, React Scan on in both
A whole race live on today's build (Overview, Current Lap): 714 MB at the end, peak 748 MB	Measured (9 October, one run)
What the 100–200 MB of renderer memory outside V8 and Blink is	Not attributed (compositor, raster, allocator)

7. Fixed costs: how much more can go

Measured on 9 October with the current build (`electron-vite` build of today's source), React Scan off, the window at its default size (60 % of the screen, 125 % scaling). Each run launches the app fresh and replays the UDP capture live on Overview, so five charts are on screen. Every change was measured in two separate sets:

- **Set 1:** one run per change, plus four baseline runs. Each figure is the median of three samples taken after 100 s. The baselines fell between 501 and 512 MB.
- **Set 2:** one run per change, plus two baseline runs, sampled every 1.5 s for 115 s. The figures are medians over the last 25 s. The baselines were 511 and 512 MB.

Totals move by up to ± 10 MB between runs, mostly in the renderer, so per-process figures are the reliable ones.

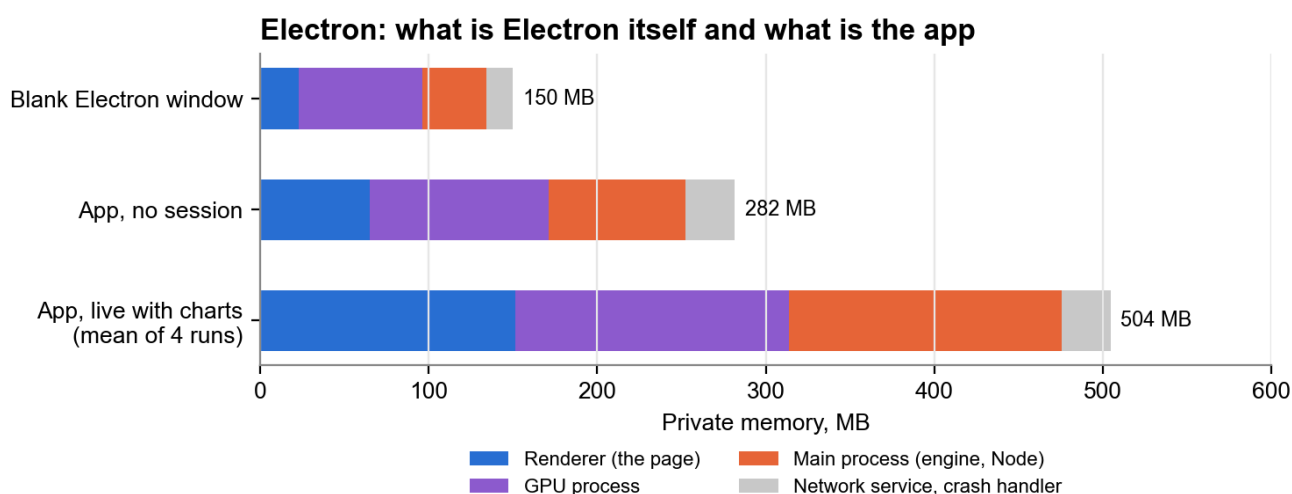


Figure 6. Private memory by process: Electron alone, the app with no session, and the app live with charts.

State	Total	Main	GPU	Renderer	Network	Crash handler
Blank Electron window	150	38	74	23	16	—
App, no session (idle)	282	81	106	65	17	13
App, live with charts (set 1, mean of 4 runs)	505	162	162	151	16	13

Three layers make up the cost:

- **Electron itself: 150 MB.** A blank Electron window with none of our code. It can't go while the app runs on Electron.
- **The app with no session: +132 MB.** The main process doubles (Node, the native addon, the engine at rest); the crash handler adds 13 MB; the renderer takes 42 MB for the React app; the GPU process 32 MB.
- **Live with charts: +222 MB more.** The main process grows by 81 MB once the engine is running. That is Strategy's rollback journal, the recording writer's lap builders and the in-memory copy of the recording (section 8, steps 1, 2 and 4). The GPU process grows by 56 MB, for 5 WebGL chart canvases and 10 2D axis canvases. The renderer grows by 86 MB for the telemetry tables, React and V8.

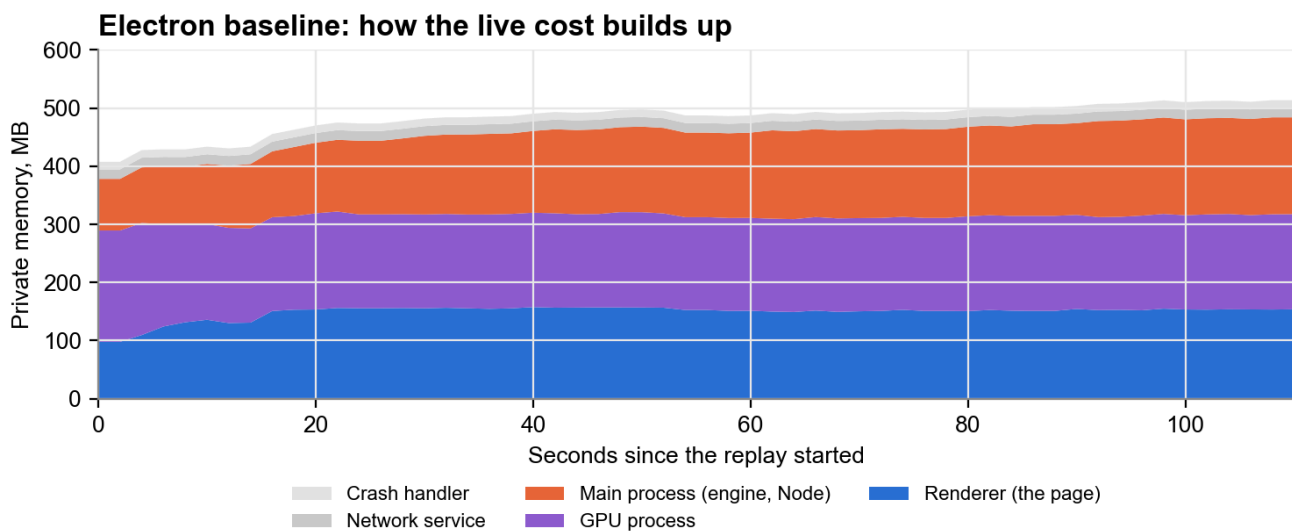


Figure 7. A baseline run over time, stacked by process (set 2). The renderer and GPU process settle within 20 s, once the charts are up. The main process keeps climbing for the whole run, from 90 to about 170 MB, as the engine fills: none of the changes below touch it.

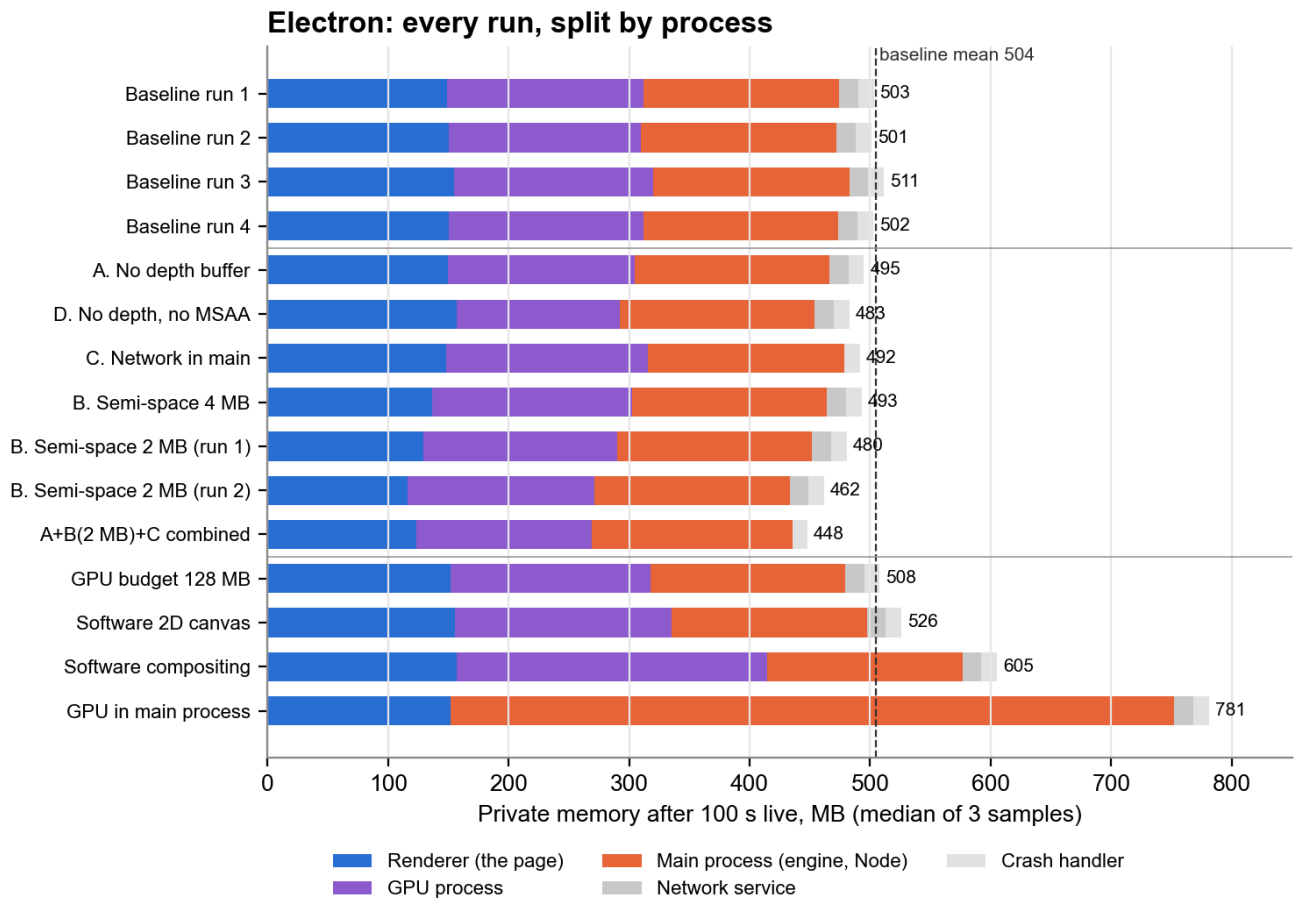


Figure 8. Every run of set 1, split by process. The four baselines are at the top; the changes worth making in the middle; the ones that cost memory at the bottom.

Electron: where each change acts (MB against the baseline mean)

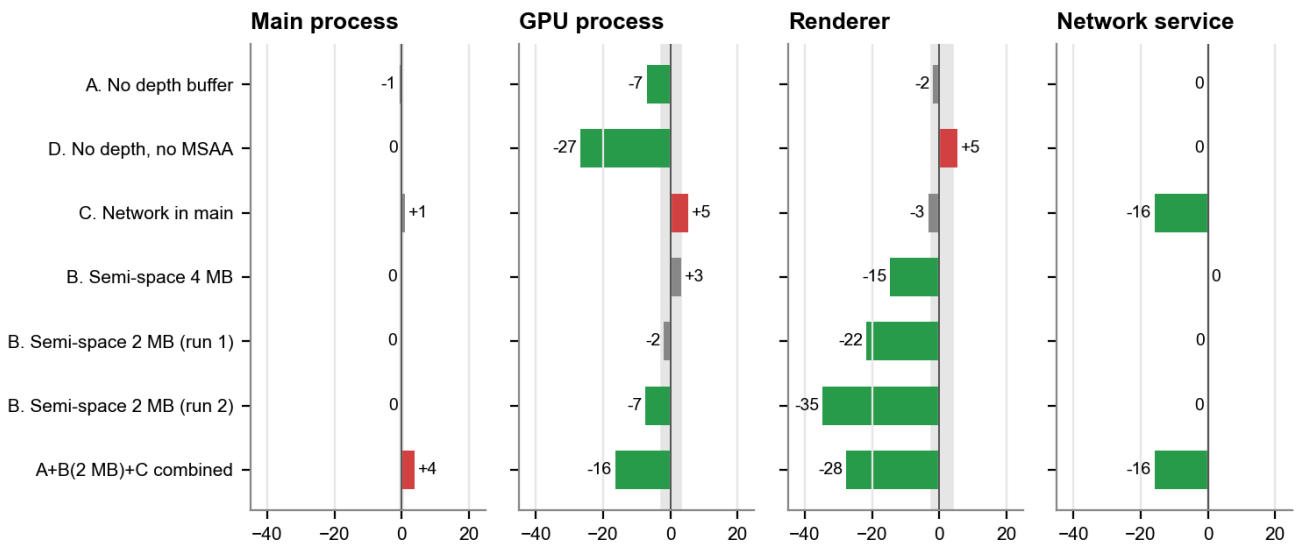


Figure 9. Where each change acts, set 1, against the mean of the four baselines. The grey band is the baseline runs' spread for that process. Each change moves only the process it should.

Electron: each change over time against the baseline band

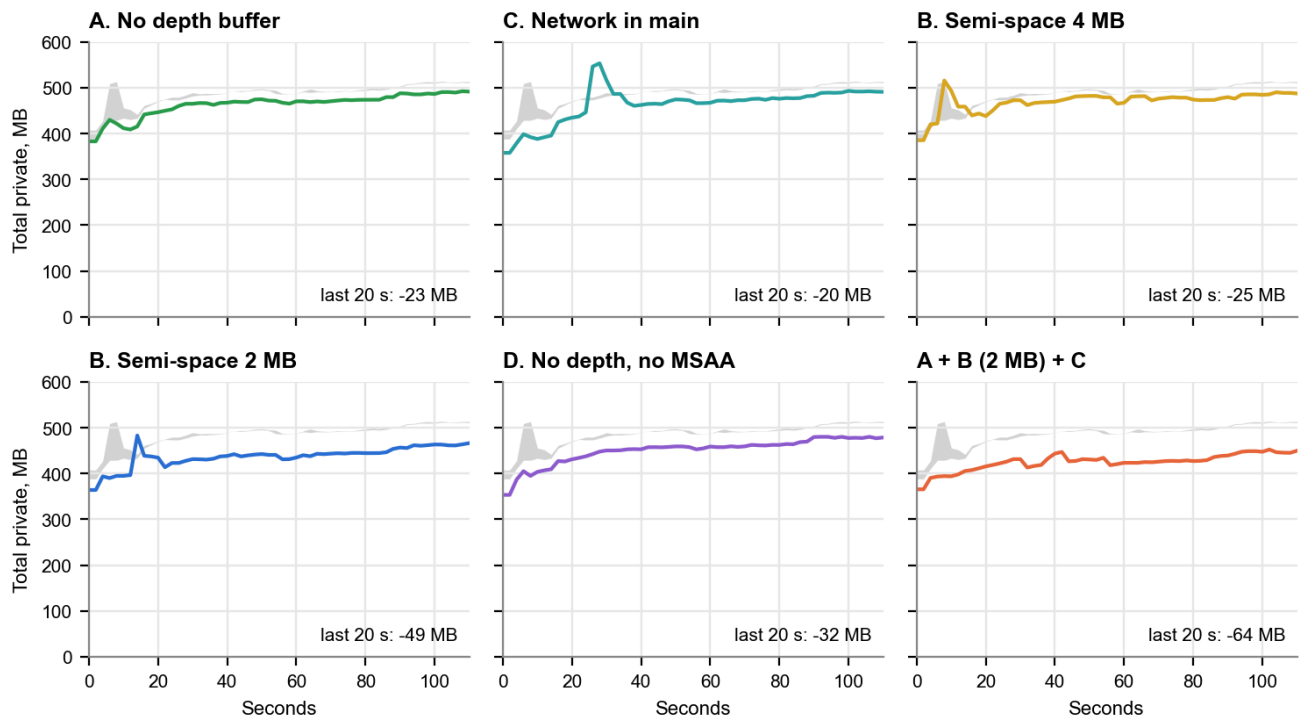


Figure 10. Set 2 over time: each change's total (coloured) against the two baseline runs (grey band). The saving holds through the whole run. The start of each run differs by a few seconds of startup.

Electron: the combined change, process by process

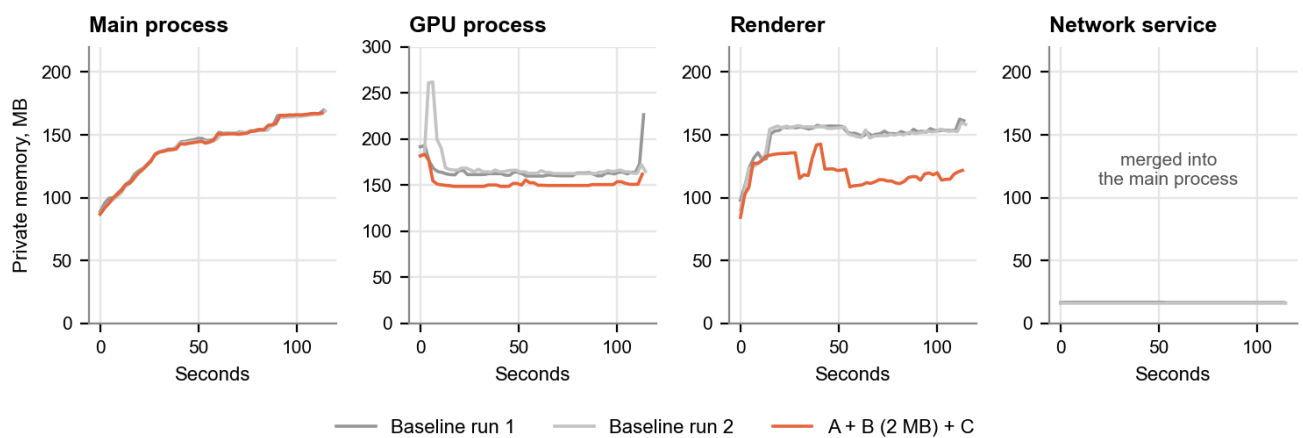


Figure 11. A + B (2 MB) + C, process by process, against the two baseline runs (set 2). The main process is unchanged; the GPU process and renderer drop as soon as the charts are up; the network service process is gone.

Change	Acts on	Set 1	Set 2	Renderer CPU
A. No depth buffer on chart canvases	GPU -7 / -13	-10	-23	no change
D. No depth buffer, no MSAA	GPU -27 / -30	-22	-33	no change
C. Network service in the main process	Network -16 / -16, main +1 / -3	-13	-21	no change
B. V8 semi-space 4 MB	Renderer -15 / -25	-11	-23	+9 % / +13 %
B. V8 semi-space 2 MB	Renderer -22 and -35 / -40	-24 and -42	-48	+17 % / +18 %
A + B (2 MB) + C	GPU, renderer, network	-56	-63	+14 % / +12 %
Smaller GPU memory budget	—	+4	not run	—
Software 2D canvas	GPU +17	+22	not run	—
Software compositing	GPU +96	+101	not run	—
GPU in the main process	Main +439	+276	not run	—

MB against each set's own baseline mean (set 1: 504.5 MB, set 2: 511.5 MB). **Acts on** shows set 1 / set 2 for the process the change targets. Renderer CPU is the increase in the renderer's CPU time over the run.

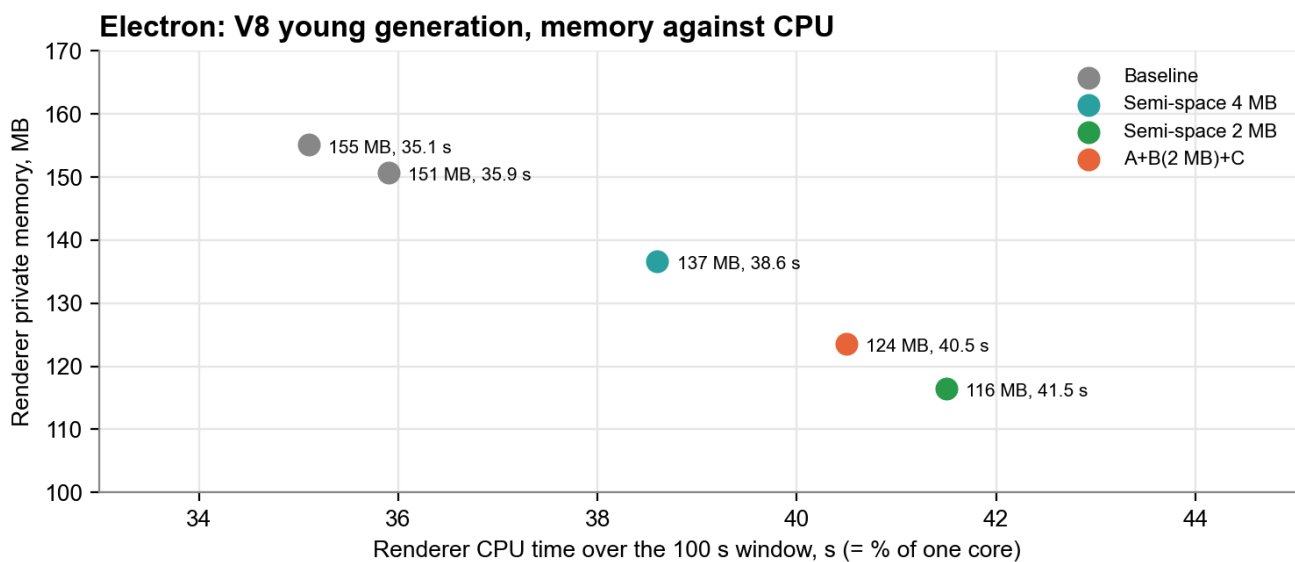


Figure 12. V8 young generation, set 1: renderer memory against renderer CPU over 100 s. Each step down in memory costs CPU.

Electron: renderer CPU with a smaller V8 young generation

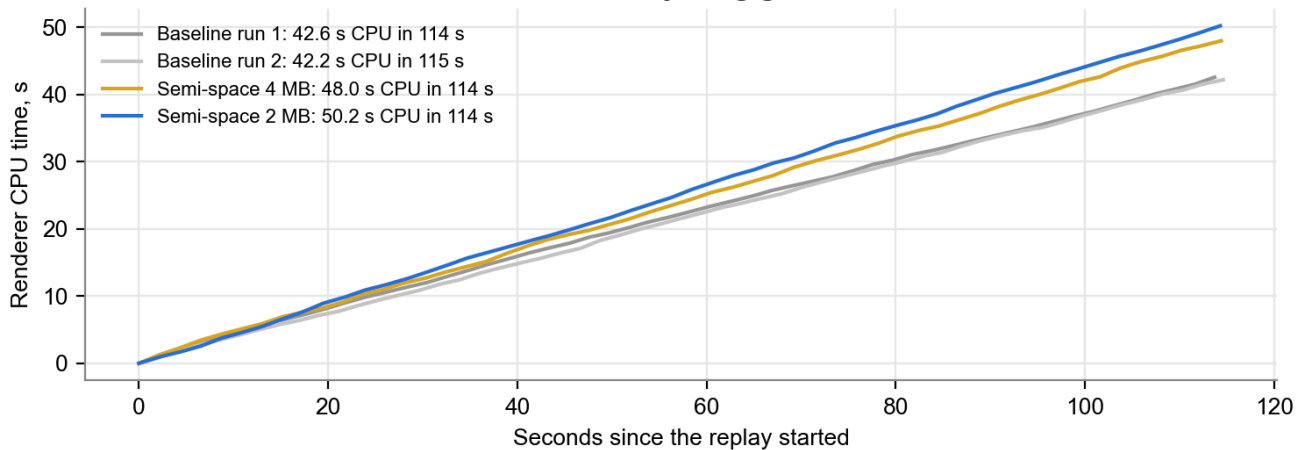


Figure 13. Renderer CPU over time, set 2. The extra CPU is a steady slope, not bursts: about +5–7 % of one core.

What to change

#	Change	Where	Saving (measured)	Cost
A	Create chart canvases with { depth: false }	canvasLayer.ts	GPU -7 to -13 MB; more with a larger window or more charts	None: nothing uses depth testing
B	Cap V8's young generation: js-flags --max-semi-space-size=4 (or 2)	main/index.ts	Renderer -15 to -25 MB (4) or -22 to -40 MB (2)	Renderer CPU +9–13 % (4) or +17–18 % (2)
C	Network service in the main process (NetworkServiceInProcess2)	main/index.ts	-13 to -21 MB	Its crash takes main down; confirm the feature name
D	No MSAA on chart canvases; antialias lines in the shader	Chart engine	GPU -27 to -30 MB	Shader work; small visual change
E	Start the crash reporter only when diagnostics are on	diagnostics.ts	-13 MB	No local crash dumps otherwise
F	Engine steps 1, 2 and 4 (section 8)	Engine	Main +140 MB over a whole race (section 5)	See section 8

A, B (2 MB) and C together saved 56 MB in set 1 and 63 MB in set 2 (11–12 %). With the gentler 4 MB V8 setting, expect about 45 MB (estimate from the single-change runs; not measured as a combination). The engine steps from section 8 would take a long session down by another 50–100 MB (estimate), mostly in the main process. They shrink Qt by the same amount, so they don't close the gap; A–C would take the live gap with Qt from about 250 MB to about 200 MB (estimate). Most of what remains is Electron itself.

Tested and not worth it: running the GPU in the main process (+276 MB), software compositing (+101 MB), drawing the 2D canvases in software (+22 MB, moved into the GPU and renderer), and a smaller GPU memory budget (no change). These were measured in set 1 only.

8. Next steps for optimization

Ordered by expected saving for the effort. None of them drops or thins out data. **Saving** says whether the figure is measured, computed or an estimate.

#	Step	Where	Saving	Effort and risk
1	Keep the recording's chunks on disk while a file holds them	Engine, main process	About 80 MB by race end (measured in Qt; same engine)	Medium. Shared with Qt
2	Shrink Strategy's rollback journal	Engine, main process	Up to ~25 MB	Low. Shared with Qt
3	Transfer the seek payload instead of copying it, and release it in main	bridgeManager.ts, preload, store	At least one 35 MB copy at the seek peak (estimate)	Low–medium
4	Reuse the parser's result buffers	Main process, addon	Less committed memory from churn (not estimated)	Medium
5	Feed the writer structs instead of JSON	Engine and addon	Less churn and CPU (not estimated)	Medium
6	Cut the remaining renderer allocation (18–25 MB/s)	React components, store subscriptions	Smaller committed V8 heap (not estimated)	Medium
7	Apply the fixed-cost changes A–C from section 7	canvasLayer.ts, src/main/index.ts	About 45–63 MB (measured; 45 estimated)	Low. +9–18 % renderer CPU for the V8 setting
8	Attribute the renderer's 100–200 MB outside V8 and Blink	Chromium memory tracing	None by itself; finds the next target	Low
9	Measure a full live race on today's build	1x replay, React Scan off	None; replaces the outdated 1.2 GB figure	Low

Notes

- **Recording chunks on disk (1) and Strategy's journal (2)** are the same engine changes as in the Qt report. Electron runs the engine in its main process with `binaryPlayback` on (`bridgeManager.ts`), so it holds the same in-memory copy of the recording, about 80 MB by the end of a race (measured in Qt's full-race run). These are the main process's only items that grow with the session.
- **The seek payload (3).** The native seek callback hands the history buffer to `broadcast`, which sends it with `webContents.send` (`bridgeManager.ts`, around line 799). That copies the payload into each window, and main's own copy lives until garbage collection. Sending it once through a `MessagePort` with the buffer transferred, then dropping main's reference, removes at least one copy of a 35 MB payload during the 1,115 MB peak. How much of the ~375 MB peak this covers is not measured; most of the rest is likely the renderer decoding into tables.
- **Allocation churn (4–6).** Parser result buffers alone were 24 GB of allocations per 50 minutes in the original log. The renderer still allocates 18–25 MB/s, mostly React re-rendering. Churn doesn't show up as data, but V8 and the allocator keep memory committed to keep up with it, which is how the renderer reached 575 MB before.
- **Fixed costs (7).** Measured in section 7: no depth buffer on chart canvases, a smaller V8 young generation and the network service in the main process save 56–63 MB together with the 2 MB V8 setting (about 45 MB with 4 MB); MSAA and the crash handler are further options with a cost.

9. Observations

Overall: acceptable, and much better than a week ago, but heavier than Qt. The telemetry is no longer the problem. What's left is mostly the cost of running Chromium.

- **The fixed cost is large, but not all of it is Electron.** Electron alone is 150 MB; the app adds 132 MB with no session and another 222 MB once live. Section 7 measured 56–63 MB that can go with three small changes (about 45 MB with the gentler V8 setting). The engine steps are the bigger lever for long sessions, but they help Qt equally, so Electron would still sit about 200 MB above Qt once live (estimate).
- **The seek peak is the number users will notice.** 1,115 MB for a few seconds while a full-race history is decoded. Most likely the payload exists in the main process, in transit and in the renderer at once (inferred, not measured). Qt's highest point in the same run is 712 MB.
- **The renderer work did what it set out to do.** Telemetry tables are about 2.8× smaller and the renderer no longer grows by itself. The 103-minute log's flat 575 MB renderer was garbage and debug-tool retention, not data.
- **The main process is the next lever.** It holds 175–310 MB. The open items from the renderer work all sit there: reuse the parser's result buffers, drop committed chunks from memory while a recording file holds them, and feed the writer structs instead of JSON. Strategy also keeps running in the background regardless.
- **A whole race on today's build is now measured:** 714 MB at the end, against 1.2 GB for the old build's session, which also had React Scan on. The main process carries the growth; the renderer and GPU process settle early.